Constant-depth circuits versus arbitrarily large polylogarithmic monotone depth for a graph property

Bruno P. Cavalar, Igor C. Oliveira

doi:10.4230/LIPIcs.CCC.2023.29; CC BY4.0

Editorial note. Original authors: Bruno P. Cavalar and Igor C. Oliveira. arXiv:2305.06821v1 (2023-05-11), https://arxiv.org/abs/2305.06821v1. Creative Commons Attribution4.0.

Editorial note. License: https://creativecommons.org/licenses/by/4.0/

Editorial note. Selected complete native author theorem/proof/context and full bibliography in standalone layout. Native input/mathhead and mathematical style definitions flattened as explicit spans; unused frontmatter and figure packages omitted. No mathematical changes or new proofs. Original CC BY4.0 license retained.

Allowed standard prerequisites (editorial).

- Known monotone lower bounds, logspace algorithms and threshold circuit bounds. Exact author prior theorem statements retained with citations. Full author depth-reduction/padding support proofs included.

Theorem 2.1 (Polynomial-size constant-depth vs. larger monotone depth)**.** *For every $k \geq 1$, we have* $\mathsf{AC}^0 \cap \mathsf{Mono} \not\subseteq \mathsf{mDEPTH}[(\log n)^k]$. *Moreover, this separation holds for a monotone graph property.*

3 Preliminaries

3.1 Notation

Boolean functions. We denote by Mono the set of all monotone Boolean functions. We define $\mathsf{poly} = \{n \mapsto n^C : C \in \mathbb{N}\}$. A Boolean function $f : \{0,1\}^{\binom{n}{2}} \to \{0,1\}$ is said to be a graph property if $f(G) = f(H)$ for any two isomorphic graphs G and H. Let $\mathcal{F} = \{f_n\}_{n\in\mathbb{N}}$ be a sequence of graph properties, where f_n is defined over undirected graphs on n vertices. We say that $\mathcal{F}$ is *preserved under homomorphisms* if, whenever there is a homomorphism from a graph G to a graph H, we have $\mathcal{F}(G) \leq \mathcal{F}(H)$. We denote by $\mathsf{HomPreserving}$ the set of all graph properties which are preserved under homomorphisms. Note that $\mathsf{HomPreserving} \subseteq \mathsf{Mono}$.

Boolean circuits. We denote by $\mathsf{AC}^0_d[s]$ the family of Boolean functions computed by size-s, depth-d Boolean circuits with unbounded fan-in $\{\wedge, \vee\}$-gates and input literals from $\{x_1, \overline{x_1}, \ldots, x_n, \overline{x_n}\}$. We write $\mathsf{AC}^0[s]$ as a shorthand for $\bigcup_{d=1}^{\infty} \mathsf{AC}^0_d[s]$, and AC^0 as a shorthand of $\mathsf{AC}^0[n^{O(1)}] = \mathsf{AC}^0[\mathsf{poly}]$. We will also refer to $\mathsf{AC}^0_d[\mathsf{poly}]$ by AC^0_d. We write $\mathsf{DNF}[s]$ to denote the family of Boolean functions computed by size-s DNFs, where size is measured by number of terms. We write $\mathsf{CNF}[s]$ analogously. We write $\mathsf{SIZE}[s]$ to denote the family of Boolean functions computed by size-s circuits. We write $\mathsf{DEPTH}[d]$ to denote the family of Boolean functions computed by fan-in 2 circuits of depth d. We denote by $\mathsf{AC}^0[\oplus]$ the family of Boolean functions computed by polynomial-size AC^0 circuits with unbounded fan-in $\oplus$-gates.

We denote by L the family of Boolean functions computed by logspace machines, and by NL the family of Boolean functions computed by polynomial-time nondeterministic logspace machines. Moreover, we denote by $\oplus\mathsf{L}$ the family of Boolean functions computed by polynomial-time nondeterministic logspace machines with a *parity* acceptance condition (i.e., an input is accepted if the number of accepting paths is odd).

Circuit complexity. Given a circuit class $\mathcal{C}$, we write $\mathsf{m}C$ to denote the monotone version of $\mathcal{C}$. Given a function f, we write $\mathsf{mSIZE}(f)$ to denote the size of the smallest monotone circuit computing f and $\mathsf{mDEPTH}(f)$ to denote the smallest depth of a fan-in 2 monotone circuit computing f. Given two Boolean functions f, g, we write $f \leq_m^{\mathsf{mProj}} g$ if there exists a many-one reduction from f to g in which each bit of the reduction is a monotone projection[1] of the input.

Miscellanea. Let $\alpha \in \{0,1\}^n$. We define $|\alpha|_1 := \sum_{i=1}^n \alpha_i$. We call $|\alpha|_1$ the *Hamming weight* of α. We let $\mathrm{supp}(\alpha) = \{i \in [n] : \alpha_i = 1\}$. We let $\mathrm{THR}_{k,n} : \{0,1\}^n \to \{0,1\}$ be the Boolean function such that $\mathrm{THR}_{k,n}(x) = 1 \iff |x|_1 \geq k$.

3.2 Background results

The next lemma, which is proved via a standard "guess-and-verify" approach, shows that nondeterministic logspace computations can be simulated by circuits of size 2^{n^ε} and of depth $d = O_\varepsilon(1)$.

Lemma 3.1 (Folklore; see, e.g., [AHM+08, Lemma 8.1])**.** *For all $\varepsilon > 0$, we have $\mathsf{NL} \subseteq \mathsf{AC}^0[2^{n^\varepsilon}]$.*

Theorem 3.2 (Polynomial-size constant-depth vs. larger monotone size)**.** *For every $k \geq 1$, we have $\mathsf{AC}^0[\oplus] \cap \mathsf{Mono} \not\subseteq \mathsf{mSIZE}[2^{(\log n)^k}]$.*

Theorem 3.3. *For every $i \geq 1$, we have $\mathsf{AC}^0 \cap \mathsf{Mono} \cap \mathsf{GraphProperties} \not\subseteq \mathsf{mDEPTH}[(\log n)^i]$. In particular, we have $\mathsf{AC}^0 \cap \mathsf{Mono} \cap \mathsf{GraphProperties} \not\subseteq \mathsf{mNC}^i$.*

Lemma 3.4 (Folklore; see, e.g., [AKR+01, OSS19])**.** *Let $f : \{0,1\}^n \to \{0,1\}$ be a Boolean function computed by a $\oplus\mathsf{L}$ machine. For every $\delta > 0$, there exists an $\mathsf{AC}^0[\oplus]$ circuit of size 2^{n^δ} that computes f.*

Proof. Let M be a $\oplus\mathsf{L}$-machine computing f. Without loss of generality, we may assume that each configuration in the configuration graph G of M is *time-stamped* – in other words, each configuration carries the information of the number of computational steps it takes to arrive at it.[2] We may also assume that every accepting computation takes exactly the same amount of time, which means that every path from the starting configuration v_{start} to the accepting configuration v_{accept} has the same length in the configuration graph. These assumptions imply that the configuration graph is *layered* (because a configuration with time-stamp t can only point to configurations with time-stamp $t+1$) and acyclic. Note that, for a fixed machine, the configuration graph can be computed from the input string using a projection.

Let $m = n^{O(1)}$ be the time that an accepting computation takes. We now show how to count (modulo 2) the number of accepting paths from v_{start} to v_{accept} with a depth-d $\mathsf{AC}^0[\oplus]$ circuit. First, choose $m^{1/d}-1$ configurations $v_1, \ldots, v_{m^{1/d}-1}$ (henceforth called "checkpoints") from $V(G)$, such that the configuration v_i is at the level $i \cdot m^{1-1/d}$ in the configuration graph (i.e., it takes $i \cdot m^{1-1/d}$ time steps to arrive at v_i). For convenience, we let $v_0 = v_{\mathsf{start}}$ and $v_{m^{1/d}} = v_{\mathsf{accept}}$. We then count the number of paths from from v_{start} to v_{accept} that go through $v_1, \ldots, v_{m^{1/d}-1}$, and sum over all possible choices of the checkpoints. Since the graph is layered and each path from v_0 to $v_{m^{1/d}}$ has length exactly m, there is only one choice of checkpoints that witnesses a given path from v_0 to $v_{m^{1/d}}$, so no path is counted twice in this summation. Letting $\#\mathsf{paths}(s,t,\ell)$ denote the number of paths between configurations s and t with distance exactly ℓ, we obtain

$$\#\mathsf{paths}(v_0, v_{m^{1/d}}, m) = \sum_{v_1,\ldots,v_{m^{1/d}-1}} \prod_{i=0}^{m^{1/d}-1} \#\mathsf{paths}(v_i, v_{i+1}, m^{1-1/d}).$$

The above calculation can be done in modulo 2 with an unbounded fan-in XOR gate (replacing the summation) and an unbounded fan-in AND gate (replacing the product). Note that the formula above is recursive. Repeating the same computation for calculating (modulo 2) the expression $\#\mathsf{paths}(v_i, v_{i+1}, m^{1-1/d})$ for each i, we obtain a depth-$2d$ $\mathsf{AC}^0[\oplus]$ circuit for calculating the number of paths from v_{start} to v_{accept} (modulo 2). Clearly, the total size of the circuit is $2^{O(m^{1/d} \cdot \log m)}$, which is smaller than 2^{n^δ} for a large enough constant d. □

[1] A monotone projection is a projection without negations.

[2] Formally, we can define a $\oplus\mathsf{L}$-machine M' such that the configurations of M' are (C,t), where C is a configuration of M, and $t = 0, 1, \ldots, m = n^{O(1)}$ is a number denoting the time in which the configuration was achieved. A configuration (C,t) can only reach a configuration $(C', t+1)$ in the configuration graph of M'.

First, we recall a result of [BGW99], which proves monotone lower bounds for the following function. Let $\mathsf{OddFactor}_n : \{0,1\}^{\binom{n}{2}} \to \{0,1\}$ be the function that accepts a given graph if it contains an *odd factor* – in other words, a spanning subgraph in which the degree of every vertex is odd. Babai, Gál and Wigderson [BGW99] proved the following result:

Theorem 3.5 ([BGW99])**.** *Any monotone formula computing* $\mathsf{OddFactor}_n$ *has size* $2^{\Omega(n)}$*, and any monotone circuit computing* $\mathsf{OddFactor}_n$ *has size* $n^{\Omega(\log n)}$*.*

The proof in [BGW99] is actually for the case of *bipartite graphs*, but it easily extends to general graphs, since the bipartite case reduces to the general case by a monotone projection. The formula lower bound stated above is slightly stronger because it makes use of asymptotically optimal lower bounds on the randomized communication complexity of DISJ_n [KS92], which were not available to [BGW99]. We remark that, with a different language, a monotone circuit lower bound for $\mathsf{OddFactor}$ is also implicitly proved in Feder and Vardi [FV98, Theorem 30].

We now recall an upper bound for $\mathsf{OddFactor}$, implicitly proved in an unpublished note due to Johannsen [Joh03].

Theorem 3.6 ([Joh03])**.** *We have* $\mathsf{OddFactor} \in \mathsf{L}$*.*

Proof. We first recall the following observation about the $\mathsf{OddFactor}$ function, which appears in different forms in the literature (see [Urq87, Lemma 4.1] or [Juk12, Lemma 18.16]; see also [Joh03, Proposition 1] for a different proof.)

Claim. *A graph G has an odd factor if and only if every connected component of G has an even number of vertices.*

Proof. If a graph G has an odd factor, we can conclude that every connected component of G has an even number of vertices from the well-known observation that in every graph there is an even number of vertices of odd degree.

Now suppose that every connected component of G has an even number of vertices. We will iteratively construct an odd factor F of G. We begin with the empty graph. We take any two vertices u, v in the same connected component of G which currently have even degree in F, and consider any path $P = (x_1, \ldots, x_k)$ between u and v, where $x_1 = u$ and $x_k = v$. If the edge $x_i x_{i+1}$ is currently in F, we remove $x_i x_{i+1}$ from F; otherwise, we add $x_i x_{i+1}$ to F. It's easy to check that, in every iteration of this procedure, only the vertices u and v have the parity of their degree changed in F; the degree of every other vertex stays the same (modulo 2). Since every connected component has an even number of vertices, this means that, eventually, every vertex in F will have odd degree. □

Now it's easy to check in logspace if every connected component of G has an even number of vertices using Reingold's algorithm for undirected connectivity [Rei05]. It suffices to check if, for every vertex v of G, the number of vertices reachable from v is odd. □

Now, if we only desire to obtain a function in AC^0 not computed by monotone circuits of depth $(\log n)^i$, we can follow the same argument of Theorem 3.2, using Lemma 3.1 instead of Lemma 3.4. In order to obtain moreover a monotone *graph property* witnessing this separation, we will need the following lemma, which enables us to obtain a graph property after "padding" a graph property. We defer the proof of this lemma to the end of this section.

Lemma 3.7. *Let $f : \{0,1\}^{\binom{n}{2}} \to \{0,1\}$ be a monotone graph property on graphs of n vertices. The following holds.*

1. *If $f \in \mathsf{NC}^i$ for some $i > 1$, then there exists a monotone graph property g on graphs of $N = 2^{(\log n)^i}$ vertices such that $g \in \mathsf{NC}^1$ and $f \leq_m^{\mathsf{mProj}} g$.*
2. *If $f \in \mathsf{NL}$, then for all $\varepsilon > 0$ there exists a monotone graph property g on graphs of $N = 2^{n^\varepsilon}$ vertices such that g can be computed by AC^0 circuits of size $N^{2+o(1)}$ and $f \leq_m^{\mathsf{mProj}} g$.*
3. *If $f \in \oplus\mathsf{L}$, then for all $\varepsilon > 0$ there exists a monotone graph property g on graphs of $N = 2^{n^\varepsilon}$ vertices such that g can be computed by $\mathsf{AC}^0[\oplus]$ circuits of size $N^{2+o(1)}$ and $f \leq_m^{\mathsf{mProj}} g$.*

We are now ready to prove Theorem 3.3.

Proof of Theorem 3.3. Fix $n \in \mathbb{N}$ and take an $\varepsilon < 1/i$. Observing that $\mathsf{L} \subseteq \mathsf{NL}$, from Theorem 3.6 and item (2) of Lemma 3.7 we conclude that there exists a monotone graph property f on $N = 2^{n^\varepsilon}$ vertices such that $f \in \mathsf{AC}^0$ and $\mathsf{OddFactor}_n \leq_m^{\mathsf{mProj}} f$. By Theorem 3.5, any monotone circuit computing f has depth $\Omega(n) = \Omega((\log N)^{1/\varepsilon}) \gg (\log N)^i$. □

Raz and Wigderson [RW92] observed that there exists a monotone function $f \in \mathsf{NC}^1 \setminus \mathsf{mNC}$. Using Lemma 3.7, we observe moreover that it's possible to obtain this separation with a monotone graph property.

Proposition 3.8. *We have* $\mathsf{NC}^1 \cap \mathsf{Mono} \cap \mathsf{GraphProperties} \not\subseteq \mathsf{mNC}$.

Proof. Observing that $\mathsf{L} \subseteq \mathsf{NC}^2$, we conclude from Theorem 3.6 and item (1) of Lemma 3.7 that there exists a monotone graph property f on $N = 2^{(\log n)^2}$ vertices such that $f \in \mathsf{NC}^1$ and $\mathsf{OddFactor}_n \leq_m^{\mathsf{mProj}} f$. By Theorem 3.5, any monotone circuit computing f has depth $\Omega(n) = \Omega(2^{\sqrt{\log N}})$, which implies $f \notin \mathsf{mNC}$. □

3.3 Efficient monotone padding for graph properties

We will now prove Lemma 3.7. We first recall some low-depth circuits for computing threshold functions, which we will use to design a circuit for efficiently computing the adjacency matrix of induced subgraphs.

Theorem 3.9 ([HWWY94])**.** *Let $d > 0$ be a constant. The function $\mathrm{THR}_{(\log n)^d, n}$ can be computed by an AC^0 circuit of size $n^{o(1)}$ and depth $d + O(1)$.*

Theorem 3.10 ([AKS83])**.** *For every $k \in [n]$, the function $\mathrm{THR}_{k,n}$ can be computed by a circuit of depth $O(\log n)$ and size $n^{O(1)}$.*

Lemma 3.11. *There exists a circuit C_n^k with $\binom{n}{2} + n$ inputs and $\binom{k}{2}$ outputs which, when given as input an adjacency matrix of a graph G on n vertices and a characteristic vector of a set $S \subseteq [n]$ such that $|S| \leq k$, outputs the adjacency matrix of the graph $G[S]$, padded with isolated vertices when $|S| < k$. The circuit has constant-depth and size $n^{2+o(1)}$ when $k = \mathrm{polylog}(n)$, and size $n^{O(1)}$ and depth $O(\log n)$ otherwise.*

Proof. Let $\{x_{ij}\}_{i,j \in [n]}$ encode the adjacency matrix of G. Let $\alpha \in \{0,1\}^n$ be the characteristic vector of S. Let $i, j \in [k]$. Note that $\{i, j\} \in E(G[S])$ if and only if there exists $a, b \in [n]$ such that

- α_a is the i-th non-zero entry of α,
- α_b is the j-th non-zero entry of α, and
- $x_{ab} = 1$ (i.e., a and b are connected in G).

We first consider the case $k = \mathrm{polylog}(n)$. In this case, the first two conditions can be checked with circuits of size $n^{o(1)}$ using Theorem 3.9. Therefore, we can compute if i and j are adjacent using $n^{2+o(1)}$ gates and constant depth. As there are at most $(\log n)^{O(1)}$ such pairs, we can output $G[S]$ with at most $n^{2+o(1)}$ gates.

For any k, the first two conditions can be checked with an NC^1 circuit by Theorem 3.10. Since there are at most n^2 pairs i, j, the entire adjacency matrix can be computed with a $O(\log n)$-depth and polynomial-size circuit. □

We are ready to prove Lemma 3.7.

Proof of Lemma 3.7. We first prove (1). Fix $n \in \mathbb{N}$ and let $N = 2^{(\log n)^i}$. For a graph G on N vertices such that $|E(G)| \leq \binom{n}{2}$, let G_{clean} be the graph obtained from G by removing isolated vertices from G one-by-one, in lexicographic order, until one of the following two conditions are satisfied: (1) there are

no more isolated vertices in G_{clean}, *or* (2) G_{clean} has exactly n vertices. Let $g : \{0,1\}^{\binom{N}{2}} \to \{0,1\}$ be the monotone graph property defined as follows:

$$g(G) := \left(|E(G)| > \binom{n}{2}\right) \vee (|V(G_{\mathsf{clean}})| > n) \vee (f(G_{\mathsf{clean}}) = 1).$$

Note that g accepts a graph G if and only if at least one of the following three conditions are satisfied:

1. G has at most $\binom{n}{2}$ edges, G_{clean} has exactly n vertices and $f(G_{\mathsf{clean}}) = 1$, or
2. G has more than $\binom{n}{2}$ edges, or
3. G_{clean} has more than n vertices.

We observe that the monotonicity of g follows from the monotonicity of f. We also claim that g is a graph property. Indeed, the graph G_{clean} is the same (up to isomorphism), irrespective of the order according to which the isolated vertices are removed from G. Moreover, the function f is also a graph property. Because of this, all the three conditions above are preserved under isomorphisms.

We first observe that f is a monotone projection of g. Indeed, given a graph G on n vertices, we can easily construct by a monotone projection a graph G' on N vertices and at most $\binom{n}{2}$ edges such that $f(G) = g(G')$. We just let G' have a planted copy of G, and all other vertices are isolated. Then $G'_{\mathsf{clean}} = G$ (up to isomorphism) and $g(G') = f(G_{\mathsf{clean}}) = f(G)$.

We now show how to compute g in NC^1. Let $\{x_{ij}\}_{i,j\in[N]}$ be the input bits of g, corresponding to the adjacency matrix of a graph G. The circuit computes as follows.

1. If $|E(G)| > \binom{n}{2}$, accept the graph G.
2. Compute the characteristic vector $\alpha \in \{0,1\}^N$ of the set of all non-isolated vertices of G. If $|\alpha|_1 > n$, accept the graph G.
3. Compute G_{clean} and output $f(G_{\mathsf{clean}})$.

Note that checking if $|E(G)| > \binom{n}{2}$ can be done in NC^1 by Theorem 3.10. Moreover, for all $i \in [N]$, we have $\alpha_i = \bigvee_{j\in[N]} x_{ij}$, and therefore α_i can be computed by a circuit of depth $O(\log N)$ and $O(N)$ gates. In total, the vector α can be computed with $O(N^2)$ gates and $O(\log N)$ depth. Finally, we can check if $|\alpha|_1 > n$ in NC^1 with a threshold circuit.

For the final step, we compute G_{clean}. If $|\alpha|_1 = n$, note that $G_{\mathsf{clean}} = G[\mathrm{supp}(\alpha)]$. When $|\alpha|_1 < n$, then G_{clean} is $G[\mathrm{supp}(\alpha)]$ padded with isolated vertices. We can therefore compute G_{clean} with the circuit C^n_N of Lemma 3.11. Moreover, since $f \in \mathsf{NC}^i$, we have that f can be computed by a circuit of size $n^{O(1)} = N^{o(1)}$ and depth $O((\log n)^i) = O(\log N)$. Therefore, computing $f(G_{\mathsf{clean}})$ can be done in NC^1. Overall, we get that $g \in \mathsf{NC}^1$.

In order to prove (2), it suffices to modify the proof above. The modification can be briefly described as follows. We let $N = 2^{n^\varepsilon}$. Every time Lemma 3.11 is applied, we use the AC^0 circuit instead of the NC^1 circuit, since $n = \mathrm{polylog}(N)$. This ammounts to $N^{2+o(1)}$ many gates with unbounded fan-in. Moreover, since by assumption $f \in \mathsf{NL}$, applying Lemma 3.1 we obtain an AC^0 circuit for f of size $2^{n^{\varepsilon/2}} = N^{o(1)}$, so we can compute $f(G_{\mathsf{clean}})$ in constant depth with $N^{o(1)}$ gates.

Finally, for (3) it suffices to apply the same argument used for (2), replacing an application of Lemma 3.1 by an application of Lemma 3.4. □

References

[AB87] Noga Alon and Ravi B. Boppana. The monotone circuit complexity of boolean functions. *Combinatorica*, 7(1):1–22, 1987.

[ABI+09] Eric Allender, Michael Bauland, Neil Immerman, Henning Schnoor, and Heribert Vollmer. The complexity of satisfiability problems: Refining Schaefer's theorem. *J. Comput. Syst. Sci.*, 75(4):245–254, 2009.

[AG87] Miklós Ajtai and Yuri Gurevich. Monotone versus positive. *J. Assoc. Comput. Mach.*, 34(4):1004–1015, 1987.

[AHM+08] Eric Allender, Lisa Hellerstein, Paul McCabe, Toniann Pitassi, and Michael E. Saks. Minimizing disjunctive normal form formulas and AC^0 circuits given a truth table. *SIAM J. Comput.*, 38(1):63–84, 2008. 2

[AK11] Jin Akiyama and Mikio Kano. *Factors and Factorizations of Graphs*, volume 2031 of *Lecture Notes in Mathematics*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.

[AKR+01] Eric Allender, Michal Koucký, Detlef Ronneburger, Sambuddha Roy, and V. Vinay. Time-space tradeoffs in the counting hierarchy. In *Proceedings of the 16th Annual IEEE Conference on Computational Complexity, Chicago, Illinois, USA, June 18-21, 2001*, pages 295–302. IEEE Computer Society, 2001. 2

[AKS83] Miklós Ajtai, János Komlós, and Endre Szemerédi. An $O(n \log n)$ sorting network. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing, 25-27 April, 1983, Boston, Massachusetts, USA*, pages 1–9. ACM, 1983. 4

[And85] Alexander E Andreev. On a method for obtaining lower bounds for the complexity of individual monotone functions. *Doklady Akademii Nauk SSSR*, 282:1033–1037, 1985.

[BCRV03] Elmar Böhler, Nadia Creignou, Steffen Reith, and Heribert Vollmer. Playing with boolean blocks, part i: Post's lattice with applications to complexity theory. *SIGACT News*, 2003.

[BCRV04] Elmar Böhler, Nadia Creignou, Steffen Reith, and Heribert Vollmer. Playing with boolean blocks, part ii: Constraint satisfaction problems. *ACM SIGACT-Newsletter*, 35, 2004.

[BDHM92] Gerhard Buntrock, Carsten Damm, Ulrich Hertrampf, and Christoph Meinel. Structure and importance of logspace-mod class. *Math. Syst. Theory*, 25(3):223–237, 1992.

[Ber82] S. J. Berkowitz. On some relationships between monotone and non-monotone circuit complexity. Technical Report. University of Toronto, 1982.

[BGW99] László Babai, Anna Gál, and Avi Wigderson. Superpolynomial lower bounds for monotone span programs. *Combinatorica*, 19(3):301–319, 1999. 3

[BHST14] Eric Blais, Johan Håstad, Rocco A. Servedio, and Li-Yang Tan. On DNF approximators for monotone boolean functions. In *International Colloquium on Automata, Languages, and Programming* (ICALP), pages 235–246, 2014.

[BL88] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *Advances in Cryptology* (CRYPTO), pages 27–35, 1988.

[BST13] Eric Blais, Dominik Scheder, and Li-Yang Tan. Ajtai-gurevich redux. Manuscript. 2013.

[BT96] Nader H. Bshouty and Christino Tamon. On the Fourier spectrum of monotone functions. *J. ACM*, 43(4):747–770, 1996.

[Bul17] Andrei A. Bulatov. A dichotomy theorem for nonuniform csps. In Chris Umans, editor, *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017, Berkeley, CA, USA, October 15-17, 2017*, pages 319–330. IEEE Computer Society, 2017.

[Bul18] Andrei A. Bulatov. Constraint satisfaction problems: complexity and algorithms. *ACM SIGLOG News*, 5(4):4–24, 2018.

[CDM21] Arkadev Chattopadhyay, Rajit Datta, and Partha Mukhopadhyay. Lower bounds for monotone arithmetic circuits via communication complexity. In *STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021*, pages 786–799. ACM, 2021.

[CHO+20] Lijie Chen, Shuichi Hirahara, Igor C. Oliveira, Ján Pich, Ninad Rajgopal, and Rahul Santhanam. Beyond Natural Proofs: Hardness Magnification and Locality. In *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, volume 151, pages 70:1–70:48, 2020.

[CKR20] Bruno Pasqualotto Cavalar, Mrinal Kumar, and Benjamin Rossman. Monotone circuit lower bounds from robust sunflowers. In *LATIN 2020: Theoretical Informatics - 14th Latin American Symposium, São Paulo, Brazil, January 5-8, 2021, Proceedings*, volume 12118 of *Lecture Notes in Computer Science*, pages 311–322. Springer, 2020.

[CKS01] Nadia Creignou, Sanjeev Khanna, and Madhu Sudan. *Complexity classifications of Boolean constraint satisfaction problems*, volume 7 of *SIAM monographs on discrete mathematics and applications*. SIAM, 2001.

[COS17] Xi Chen, Igor C. Oliveira, and Rocco A. Servedio. Addition is exponentially harder than counting for shallow monotone circuits. In *STOC'17—Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, pages 1232–1245. ACM, New York, 2017.

[CZ16] Eshan Chattopadhyay and David Zuckerman. Explicit two-source extractors and resilient functions. In *Symposium on Theory of Computing* (STOC), pages 670–683, 2016.

[dRGR22] Susanna F. de Rezende, Mika Göös, and Robert Robere. Guest column: Proofs, circuits, and communication. *SIGACT News*, 53(1):59–82, 2022.

[FV98] Tomás Feder and Moshe Y. Vardi. The computational structure of monotone monadic SNP and constraint satisfaction: A study through datalog and group theory. *SIAM J. Comput.*, 28(1):57–104, 1998. 3

[GGLR98] Oded Goldreich, Shafi Goldwasser, Eric Lehman, and Dana Ron. Testing monotonicity. In *Symposium on Foundations of Computer Science,* (FOCS), pages 426–435, 1998.

[GI12] Oded Goldreich and Rani Izsak. Monotone circuits: One-way functions versus pseudorandom generators. *Theory Comput.*, 8(1):231–238, 2012.

[GJW18] Mika Göös, Rahul Jain, and Thomas Watson. Extension complexity of independent set polytopes. *SIAM J. Comput.*, 47(1):241–269, 2018.

[GKRS19] Mika Göös, Pritish Kamath, Robert Robere, and Dmitry Sokolov. Adventures in monotone complexity and TFNP. In *10th Innovations in Theoretical Computer Science*, volume 124 of *LIPIcs. Leibniz Int. Proc. Inform.*, pages Art. No. 38, 19. Schloss Dagstuhl. Leibniz-Zent. Inform., Wadern, 2019.

[GMOR15] Siyao Guo, Tal Malkin, Igor C. Oliveira, and Alon Rosen. The power of negations in cryptography. In *Theory of Cryptography Conference* (TCC), pages 36–65, 2015.

[GS92] Michelangelo Grigni and Michael Sipser. Monotone complexity. In *Proceedings of the London Mathematical Society Symposium on Boolean Function Complexity*, page 57–75, USA, 1992. Cambridge University Press.

[Hås86] Johan Håstad. Almost optimal lower bounds for small depth circuits. In *Proceedings of the 18th Annual ACM Symposium on Theory of Computing, May 28-30, 1986, Berkeley, California, USA*, pages 6–20, 1986.

[Hir22] Shuichi Hirahara. NP-hardness of learning programs and partial MCSP. In *Symposium on Foundations of Computer Science* (FOCS), 2022.

[HWWY94] Johan Håstad, Ingo Wegener, Norbert Wurm, and Sang-Zin Yi. Optimal depth, very small size circuits for symmetric functions in AC^0. *Inf. Comput.*, 108(2):200–211, 1994. 4

[IKL+19] Christian Ikenmeyer, Balagopal Komarath, Christoph Lenzen, Vladimir Lysikov, Andrey Mokhov, and Karteek Sreenivasaiah. On the complexity of hazard-free circuits. *J. ACM*, 66(4):25:1–25:20, 2019.

[JCG97] Peter Jeavons, David Cohen, and Marc Gyssens. Closure properties of constraints. *J. ACM*, 44(4):527–548, jul 1997.

[Jea98] Peter Jeavons. On the algebraic structure of combinatorial problems. *Theor. Comput. Sci.*, 200(1-2):185–204, 1998.

[JLL76] Neil D. Jones, Y. Edmund Lien, and William T. Laaser. New problems complete for nondeterministic log space. *Math. Syst. Theory*, 10:1–17, 1976.

[Joh03] Jan Johannsen. The complexity of satisfiability problems with two occurrences. Unpublished Manuscript, 2003. 3

[Juk12] Stasys Jukna. *Boolean Function Complexity - Advances and Frontiers*. Springer, 2012. 3

[Kra97] Jan Krajíček. Interpolation theorems, lower bounds for proof systems, and independence results for bounded arithmetic. *J. Symbolic Logic*, 62(2):457–486, 1997.

[KS92] Bala Kalyanasundaram and Georg Schnitger. The probabilistic communication complexity of set intersection. *SIAM J. Discret. Math.*, 5(4):545–557, 1992. 3

[Kup21] Denis Kuperberg. Positive first-order logic on words. In *Symposium on Logic in Computer Science* (LICS), pages 1–13, 2021.

[Kup22] Denis Kuperberg. Positive first-order logic on words and graphs. *CoRR*, abs/2201.11619, 2022.

[KW93] Mauricio Karchmer and Avi Wigderson. On span programs. In *Structure in Complexity Theory Conference* (CCC), pages 102–111, 1993.

[LT09] Benoît Larose and Pascal Tesson. Universal algebra and hardness results for constraint satisfaction problems. *Theor. Comput. Sci.*, 410(18):1629–1647, 2009.

[Oko82] E. A. Okol'nishnikova. The effect of negations on the complexity of realization of monotone Boolean functions by formulas of bounded depth. *Metody Diskret. Analiz.*, (38):74–80, 1982.

[Oli15] Igor C. Oliveira. Unconditional lower bounds in complexity theory, 2015. (PhD Thesis, Columbia University).

[OSS19] Igor C. Oliveira, Rahul Santhanam, and Srikanth Srinivasan. Parity helps to compute majority. In *34th Computational Complexity Conference, CCC 2019, July 18-20, 2019, New Brunswick, NJ, USA*, pages 23:1–23:17, 2019. 2

[Pos41] Emil L. Post. *The Two-Valued Iterative Systems of Mathematical Logic. (AM-5)*. Princeton University Press, 1941.

[PR17] Toniann Pitassi and Robert Robere. Strongly exponential lower bounds for monotone computation. In *Symposium on Theory of Computing* (STOC), pages 1246–1255, 2017.

[Pud97] Pavel Pudlák. Lower bounds for resolution and cutting plane proofs and monotone computations. *J. Symb. Log.*, 62(3):981–998, 1997.

[Qui53] W. V. Quine. Two theorems about truth functions. *Bol. Soc. Mat. Mexicana*, 10:64–70, 1953.

[Raz85a] A. A. Razborov. Lower bounds on the monotone complexity of some Boolean functions. *Dokl. Akad. Nauk SSSR*, 281(4):798–801, 1985.

[Raz85b] Alexander A Razborov. Lower bounds on monotone complexity of the logical permanent. *Mathematical Notes of the Academy of Sciences of the USSR*, 37(6):485–493, 1985.

[Rei05] Omer Reingold. Undirected st-connectivity in log-space. In Harold N. Gabow and Ronald Fagin, editors, *Proceedings of the 37th Annual ACM Symposium on Theory of Computing, Baltimore, MD, USA, May 22-24, 2005*, pages 376–385. ACM, 2005. 3

[Rez23] Susanna Rezende. Personal communication, 2023.

[RM99] Ran Raz and Pierre McKenzie. Separation of the monotone NC hierarchy. *Combinatorica*, 19(3):403–435, 1999.

[Ros08a] Benjamin Rossman. Homomorphism preservation theorems. *J. ACM*, 55(3):Art. 15, 53, 2008.

[Ros08b] Benjamin Rossman. On the constant-depth complexity of k-clique. In *Symposium on Theory of Computing* (STOC), pages 721–730, 2008.

[Ros17a] Benjamin Rossman. An entropy proof of the switching lemma and tight bounds on the decision-tree size of AC^0. 2017.

[Ros17b] Benjamin Rossman. An improved homomorphism preservation theorem from lower bounds in circuit complexity. In *Innovations in Theoretical Computer Science Conference* (ITCS), pages 27:1–27:17, 2017.

[RPRC16] Robert Robere, Toniann Pitassi, Benjamin Rossman, and Stephen A. Cook. Exponential lower bounds for monotone span programs. In *IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA*, pages 406–415. IEEE Computer Society, 2016.

[RW92] Ran Raz and Avi Wigderson. Monotone circuits for matching require linear depth. *J. ACM*, 39(3):736–744, 1992. 4

[Sch78] Thomas J. Schaefer. The complexity of satisfiability problems. In *Proceedings of the 10th Annual ACM Symposium on Theory of Computing, May 1-3, 1978, San Diego, California, USA*, pages 216–226. ACM, 1978.

[Sto95] Alexei P. Stolboushkin. Finitely monotone properties. In *Symposium on Logic in Computer Science* (LICS), pages 324–330, 1995.

[Tar88] É. Tardos. The gap between monotone and nonmonotone circuit complexity is exponential. *Combinatorica*, 8(1):141–142, 1988.

[Urq87] Alasdair Urquhart. Hard examples for resolution. *J. ACM*, 34(1):209–219, 1987. 3

[Val80] Leslie G. Valiant. Negation can be exponentially powerful. *Theor. Comput. Sci.*, 12:303–314, 1980.

[Zhu17] Dmitriy Zhuk. A proof of CSP dichotomy conjecture. In Chris Umans, editor, *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017, Berkeley, CA, USA, October 15-17, 2017*, pages 331–342. IEEE Computer Society, 2017.